Lab Assignment 5

Course: Higher Level Programming Language Lab

Course Code: CSEL-3204

Submitted To

Dr. MD. Manowarul Islam

Professor

Department of CSE

Jagannath University

Submitted By

Maisha Binte Monir

ID: B210305016

Session: 2021-22 (CSE-13)

1. **ATM Machine Users cannot directly access bank server data. They interact through buttons/options only. Suppose one day, Maisha goes to a ATM booth to withdraw 20000tk from her bank account. But there is not sufficient balance message pops up. So she deposits 10000tk in her account**

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self._balance = balance #protected variable

    def get_balance(self):
        return self._balance

    def _update_balance(self, amount):
        self._balance += amount

class ATM(BankAccount):
    def deposit(self, amount):
            self._balance += amount
            print("Deposit Successful:", amount)
            print("Remaining Balance:", self._balance)

    def withdraw(self, amount):
        if amount > self._balance:
            print("There is not sufficient balance")
            print("Available Balance:", self._balance)
        else:
            self._balance -= amount
            print("Withdrawal Successful:", amount)
            print("Remaining Balance:", self._balance)

maisha_account = ATM("Maisha", 15000)
while True:
```

```
    print("\n===== ATM MENU =====")
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        print("Current Balance:", maisha_account.get_balance())

    elif choice == "2":
        amount = int(input("Enter deposit amount: "))
        maisha_account.deposit(amount)

    elif choice == "3":
        amount = int(input("Enter withdraw amount: "))
        maisha_account.withdraw(amount)

    elif choice == "4":
        print("Thank you for using ATM. Goodbye!")
        break

    else:
        print("Invalid choice. Try again.")
```

Output:

===== ATM MENU =====

1. Check Balance

2. Deposit

3. Withdraw

4. Exit

Enter your choice: 3

Enter withdraw amount: 20000

There is not sufficient balance

Available Balance: 15000

===== ATM MENU =====

1. Check Balance

2. Deposit

3. Withdraw

4. Exit

Enter your choice: 2

Enter deposit amount: 10000

Deposit Successful: 10000

Remaining Balance: 25000

===== ATM MENU =====

1. Check Balance

2. Deposit

3. Withdraw

4. Exit

Enter your choice: 4

Thank you for using ATM. Goodbye!

2. **Student Result System GPA should be updated only through authorized methods. Suppose a student Maisha is trying to view her 3.1 semester's result online through options of 1.1, 1.2, 2.1, 2.2, 3.1, 3.2, 4.1, 4.4. Only admin can update the result. Results of semesters that have not been published will show appropriate message**

```
class StudentResult:
    def __init__(self, name):
        self.name = name
```

```
        self.__gpa = {
            "1.1": 3.93, "1.2": 3.89,
            "2.1": 3.87, "2.2": 4.00,
            "3.1": 3.80, "3.2": None,
            "4.1": None, "4.2": None
        }

    def view_result(self, semester):
        if semester in self.__gpa:
            if self.__gpa[semester] is None:
                print("Result not published yet for", semester)
            else:
                print(f"{semester} GPA:", self.__gpa[semester])
        else:
            print("Invalid semester")

    def _update_gpa(self, semester, gpa):
        self.__gpa[semester] = gpa


class Admin:
    def __init__(self, system):
        self.system = system

    def update_result(self, student, semester, gpa):
        student._update_gpa(semester, gpa)
        print("Result updated successfully")


maisha = StudentResult("Maisha")
admin = Admin("System")
```

```
admin.update_result(maisha, "3.1", 3.96)


print("\n--- Student Access ---")
sem = input('Enter semester to view result: ')
maisha.view_result(sem)
```

Output:

Result updated successfully

--- Student Access ---

Enter semester to view result: 2.2

2.2 GPA: 4.0

3. **Online Shopping Cart Product price calculation is hidden from users. The system automatically applies:**
   Product price × quantity
   Discount (if applicable)
   Final total calculation is hidden inside the system.

Maisha is shopping online and adds: 2 items of "Keyboard" (1200 each) and 1 item of "Mouse" (800 each)

```
class ShoppingCart:
    def __init__(self):
        self.__cart = []

    def add_product(self, name, price, quantity):
        self.__cart.append((name, price, quantity))

    def __calculate_total(self):
        total = 0
        for name, price, quantity in self.__cart:
            total += price * quantity
```

```
        return total

    def show_bill(self):
        for name, price, quantity in self.__cart:
            print(name, "x", quantity, "=", price * quantity)

        print("------------------------")
        print("Total Payable:", self.__calculate_total())

maisha_cart = ShoppingCart()
maisha_cart.add_product("Keyboard", 1200, 2)
maisha_cart.add_product("Mouse", 800, 1)
maisha_cart.show_bill()
```

Output:

Keyboard x 2 = 2400

Mouse x 1 = 800

Total Payable: 3200

4. **Hospital Management System Patient records are protected from unauthorized access. Rahim visits a hospital and his medical record includes:**
   Diagnosis history,
   Prescribed medicines,
   Test results,
   Billing information.

However, only authorized doctors or admin staff can view or update records and patients cannot directly modify or access internal medical database logic

```
class PatientRecord:
    def __init__(self, name):
        self.name = name
        self.__medical_history = []
```

```
    def add_record(self, doctor, record):
        self.__medical_history.append((doctor, record))
        print("Record added by", doctor)

    def view_record(self, role):
        if role == "doctor" or role == "admin":
            print("Medical History for", self.name)
            for doc, rec in self.__medical_history:
                print(doc, ":", rec)
        else:
            print("Access Denied: Unauthorized user")

p1 = PatientRecord("Rahim")
p1.add_record("Dr. Islam", "Fever - Paracetamol prescribed")
p1.add_record("Dr. Khan", "Blood test normal")

print("\nAccessing...")
p1.view_record("patient")
p1.view_record("doctor")
```

Output:

Record added by Dr. Islam

Record added by Dr. Khan

Accessing...

Access Denied: Unauthorized user

Medical History for Rahim

Dr. Islam : Fever - Paracetamol prescribed

Dr. Khan : Blood test normal

5. **Suppose Maisha went to the bank to withdraw 5000Tk but her account only has 5500 Tk. The bank has rules such that if the withdrawn amount results in**

balance less than 1000 Tk, the bank issues a penalty charge of 10 Tk. Also, if the account holder deposits more than 2000 Tk at once, the bank gives a bonus of 2% for every deposit. Now show the implementation of such system

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid deposit amount!")
            return

        bonus = 0
        if amount > 2000:
            bonus = amount * 0.02  # 2% bonus

        self.__balance += amount + bonus

        print(f"Deposited: {amount} Tk")
        if bonus > 0:
            print(f"Bonus Added: {bonus:.2f} Tk")
        print(f"Current Balance: {self.__balance:.2f} Tk")

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient Balance!")
            return

        self.__balance -= amount
        print(f"Withdrawn: {amount} Tk")
```

```
        # Penalty if balance becomes less than 1000 Tk
        if self.__balance < 1000:
            self.__balance -= 10
            print("Penalty Charge Applied: 10 Tk")
        print(f"Current Balance: {self.__balance:.2f} Tk")

    def get_balance(self):
        return self.__balance

account = BankAccount("Maisha", 5500)
print("Current Balance:", account.get_balance(), "Tk")
print("\n--- Withdrawal ---")
account.withdraw(5000)
print("--- Deposit ---")
account.deposit(3000)
print("Final Balance:", account.get_balance(), "Tk")
```

Output:

Current Balance: 5500 Tk

--- Withdrawal ---

Withdrawn: 5000 Tk

Penalty Charge Applied: 10 Tk

Current Balance: 490.00 Tk

--- Deposit ---

Deposited: 3000 Tk

Bonus Added: 60.00 Tk

Current Balance: 3550.00 Tk

Final Balance: 3550.0 Tk

6. **An online shopping platform allows customers to pay using different payment methods such as Credit Card and Bitcoin. Although all payment methods must perform the common task of processing a payment, the actual implementation differs: A Credit Card payment requires validating the card number and charging the amount. A Bitcoin payment requires verifying the wallet address and transferring cryptocurrency. The checkout system should not need to know the internal details of each payment method. It should simply call a common method to process the payment.**
   Implement this system using Data Abstraction in Python by creating an abstract class PaymentMethod with an abstract method process_payment(). Then create concrete subclasses CreditCard and Bitcoin that provide their own implementations.

```
from abc import ABC, abstractmethod
class PaymentMethod(ABC):

    @abstractmethod
    def process_payment(self, amount):
        pass

class CreditCard(PaymentMethod):
    def __init__(self, card_number):
        self.card_number = card_number

    def process_payment(self, amount):
        print("Credit Card Payment")
        print(f"Validating card: {self.card_number}")
        print(f"Processing payment of {amount} Tk")
        print("Payment Successful!\n")

class Bitcoin(PaymentMethod):
    def __init__(self, wallet_address):
        self.wallet_address = wallet_address

    def process_payment(self, amount):
```

```
        print("Bitcoin Payment")
        print(f"Verifying wallet: {self.wallet_address}")
        print(f"Transferring Bitcoin equivalent of {amount} Tk")
        print("Transaction Confirmed!\n")

class Checkout:
    def make_payment(self, payment_method, amount):
        payment_method.process_payment(amount)

checkout = Checkout()
credit = CreditCard("1234-5678-9012-3456")
bitcoin = Bitcoin("1A2B3C4D5E6F")

checkout.make_payment(credit, 5000)
checkout.make_payment(bitcoin, 8000)
```

Output:

Credit Card Payment

Validating card: 1234-5678-9012-3456

Processing payment of 5000 Tk

Payment Successful!

Bitcoin Payment

Verifying wallet: 1A2B3C4D5E6F

Transferring Bitcoin equivalent of 8000 Tk

Transaction Confirmed!